Contents

Topizzy — Airtime.sol Security Audit Findings

Auditor: Allan Robinson
Repository: https://github.com/FredMunene/topizzy
Scope: `topizzy/smart_contracts/src/Airtime.sol`
Date: 2026-05-08
Solidity Version: `^0.8.13`
Chain: Base (EVM-compatible)
Protocol: USDC payment gateway for airtime top-ups across East Africa

---

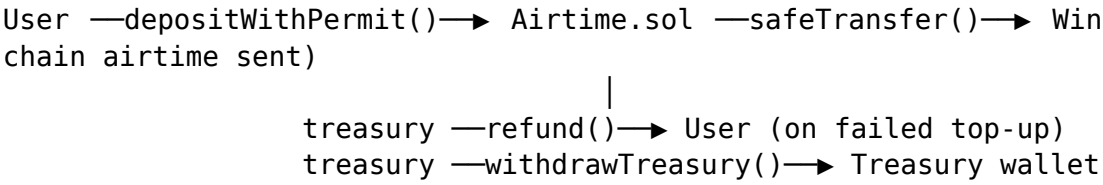
Summary Table

ID	Title	Severity
H-1	No refund deduplication — treasury can drain contract by refunding the same order multiple times	**High**
H-2	Single EOA treasury has unrestricted instant access to all user deposits	**High**

ID	Title	Severity
M-1	No treasury transfer mechanism — key loss permanently locks all user deposits	**Medium**
M-2	`withdrawTreasury()` missing `nonReentrant` guard — inconsistent with `refund()`	**Medium**
M-3	EIP-2612 permit front-running griefing forces users onto standard approval path	**Medium**
L-1	`depositRef` parameter not validated — empty or duplicate references break off-chain order matching	**Low**
L-2	`depositCounter` and returned `depositId` provide no on-chain utility	**Low**
I-1	Test suite is incompatible with current contract interface — zero test coverage	**Informational**
I-2	SPDX-License-Identifier set to UNLICENSED	**Informational**
I-3	No NatSpec documentation on any public function	**Informational**

Contract Overview

Airtime.sol is a USDC payment gateway deployed on Base. Users deposit USDC to purchase airtime via an off-chain backend (Africa's Talking API). A privileged `treasury` address can issue refunds to users on failed top-ups and withdraw accumulated USDC. The contract uses EIP-2612 permits for gasless deposits and OpenZeppelin's `ReentrancyGuard` and `SafeERC20`.



High Severity

[H-1] No refund deduplication — treasury can drain contract by refunding the same order multiple times

Severity: High
Likelihood: Medium
Impact: High

Description The `refund()` function takes an `orderRef` string, a `receiver` address, and an `amount` to transfer. However, the contract does **not record which orders have already been refunded**. There is no mapping from `orderRef` to a refunded flag, and no check that prevents the same `orderRef` from being processed more than once.

```
// src/Airtime.sol#L79-L86
function refund(string memory orderRef, address receiver, uint256 amount)
    external onlyTreasury nonReentrant
{
    require(receiver != address(0), "Invalid receiver");
    require(amount > 0, "Amount must be > 0");

    IERC20(usdcToken).safeTransfer(receiver, amount);  // no check: was this orderRef a

    emit Refunded(orderRef, receiver, amount);
}
```

The entire deduplication responsibility is delegated to the off-chain backend database. This is a critical trust assumption: if the backend has a bug, is attacked, or the treasury key is used by a malicious actor, `refund()` can be called repeatedly with the same `orderRef` until the contract's entire USDC balance reaches zero.

Impact

- A backend bug that fires the same refund webhook twice drains a real user's funds from the pool.
- A compromised treasury key allows an attacker to call `refund("any-ref", attacker, balance)` once and take everything — or loop until fully drained.
- Other users' deposited funds are at risk from any single refund operation gone wrong.

Proof of Concept Forge unit test — `test/audit/AirtimeAudit.t.sol::test_doubleRefundDrain`

```
function test_doubleRefundDrainsOtherUsersFunds() public {
    // Three users deposit 100 USDC each — 300 USDC total in contract
    // userA's top-up fails — treasury correctly issues one refund
    airtime.refund("ORDER-A", userA, 100e18);
```

```
    // BUG: same ORDER-A refunded again (backend bug or compromised key)
    // Contract has NO check — processes it without reverting
    airtime.refund("ORDER-A", userA, 100e18);

    // userA received 200 USDC from a 100 USDC deposit
    // userB and userC's funds drained to cover the double refund
    assertEq(usdc.balanceOf(userA), 200e18);
    assertEq(usdc.balanceOf(address(airtime)), 100e18);
}
```

Unit test result:

```
[PASS] test_doubleRefundDrainsOtherUsersFunds() (gas: 263498)
Logs:
  ---------- H-1: DOUBLE REFUND ----------
  Contract USDC balance before (wei): 300000000000000000000
  Contract USDC balance after  (wei): 100000000000000000000
  userA received (wei)               : 200000000000000000000
  CONFIRMED: Same orderRef refunded twice. Other users lost funds.
```

Run with:

```
cd topizzy/smart_contracts
FOUNDRY_PROFILE=audit forge test --match-test test_doubleRefundDrainsOtherUsersFunds -v
```

Live Anvil demo — `script/AttackDoubleRefund.sol`

Deploy MockUSDC + Airtime, fund 3 users, execute double refund:

```
forge script script/AttackDoubleRefund.sol --tc AttackDoubleRefund \
  --rpc-url http://127.0.0.1:8545 --broadcast
```

Script output:

```
=== H-1: DOUBLE REFUND SETUP ===
Airtime contract : 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
MockUSDC         : 0x5FbDB2315678afecb367f032d93F642f64180aa3
Treasury (owner) : 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266
UserA            : 0x70997970C51812dc3A010C7d01b50e0d17dc79C8
UserB            : 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC
UserC            : 0x90F79bf6EB2c4f870365E785982E1f101E93b906

---------- BEFORE ATTACK ----------
Contract USDC balance : 300 USDC (3 users x 100)
UserA USDC balance    : 0 (deposited)

--- AFTER LEGITIMATE REFUND (ORDER-A, first time) ---
UserA should have     : 100 USDC

---------- AFTER DOUBLE REFUND ----------
UserA USDC balance    : 200 USDC (paid TWICE for one order)
```

```
Contract USDC balance : 100 USDC (userB or userC funds stolen)

CONFIRMED: refund() accepted ORDER-A a second time.
No on-chain check exists. Other users' deposits cover the duplicate.
```

Verify damage with `cast` after the script:

```
# Contract holds only 100 USDC — down from 300
cast call 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  "balanceOf(address)(uint256)" 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512 \
  --rpc-url http://127.0.0.1:8545
# 100000000000000000000 [1e20]  ← 100 USDC (was 300)

# UserA received 200 USDC from a 100 USDC deposit
cast call 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  "balanceOf(address)(uint256)" 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 \
  --rpc-url http://127.0.0.1:8545
# 200000000000000000000 [2e20]  ← 200 USDC (should be 100)

# UserB balance — 0, their deposit absorbed the theft
cast call 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  "balanceOf(address)(uint256)" 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC \
  --rpc-url http://127.0.0.1:8545
# 0

# UserC balance — 0
cast call 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  "balanceOf(address)(uint256)" 0x90F79bf6EB2c4f870365E785982E1f101E93b906 \
  --rpc-url http://127.0.0.1:8545
# 0
```

On-chain damage summary:

Address	Role	USDC Before	USDC After
`0xe7f1...0512`	Airtime contract	300	**100**
`0x7099...79C8`	UserA (double-refunded)	0	**200**
`0x3C44...93BC`	UserB (victim)	0	**0**
`0x90F7...b906`	UserC (victim)	0	**0**

UserB and UserC each deposited 100 USDC expecting airtime. Instead, their funds silently absorbed the duplicate refund. They have no airtime, no USDC, and no way to recover.

Recommended Mitigation Track refund state on-chain using a mapping. Mark each `orderRef` as refunded before transferring to prevent any double processing:

```
+ mapping(bytes32 => bool) private _refunded;
```

```
function refund(string memory orderRef, address receiver, uint256 amount)
    external onlyTreasury nonReentrant
{
    require(receiver != address(0), "Invalid receiver");
    require(amount > 0, "Amount must be > 0");

+   bytes32 refKey = keccak256(abi.encodePacked(orderRef));
+   require(!_refunded[refKey], "Airtime: order already refunded");
+   _refunded[refKey] = true;    // mark BEFORE transfer (CEI)

    IERC20(usdcToken).safeTransfer(receiver, amount);
    emit Refunded(orderRef, receiver, amount);
}
```

Similarly, track deposits by `depositRef` to prevent accidental double deposits and make on-chain state auditable:

```
mapping(bytes32 => uint256) public depositsByRef;  // depositRef => amount deposited

// in depositWithPermit / deposit:
bytes32 depKey = keccak256(abi.encodePacked(depositRef));
require(depositsByRef[depKey] == 0, "Airtime: deposit ref already used");
depositsByRef[depKey] = amount;
```

[H-2] Single EOA treasury has unrestricted instant access to all user deposits

Severity: High
Likelihood: Medium
Impact: High

Description The `treasury` address is a single externally-owned account (EOA) that can:

1. Withdraw **any amount** of USDC at any time via `withdrawTreasury()`
2. Issue **arbitrary refunds** to any address via `refund()`
3. Do both with **no timelock**, **no spending limit**, and **no multi-sig requirement**

```
// src/Airtime.sol#L88-L95
function withdrawTreasury(address receiver, uint256 amount) external onlyTreasury {
    require(receiver != address(0), "Invalid receiver");
    require(amount > 0, "Amount must be > 0");

    IERC20(usdcToken).safeTransfer(receiver, amount);  // instant, unlimited, no delay

    emit TreasuryWithdrawal(receiver, amount);
}
```

All deposited user funds are secured by the secrecy of a **single private key** — the same key that the README confirms is stored in a `.env` file (`TREASURY_PRIVATE_KEY=0x...`). A leaked or stolen `.env` file means instant, total loss of all user deposits.

Impact

- Treasury key compromise → attacker calls `withdrawTreasury(attacker, totalBalance)` in a single transaction — all user USDC drained instantly.
- No waiting period, no alert window, no way to pause or recover.
- Users who have deposited USDC trusting the "automatic refund" mechanism have no on-chain protection.

Proof of Concept Forge test — `test/audit/AirtimeAudit.t.sol::test_treasuryDrainsAllUser`

```
function test_treasuryDrainsAllUserFunds() public {
    // Five users deposit a total of 1500 USDC
    // Treasury (or attacker with treasury key) calls withdrawTreasury once
    uint256 contractBalance = usdc.balanceOf(address(airtime));
    airtime.withdrawTreasury(treasury, contractBalance);

    // All 1500 USDC gone in a single transaction — no timelock, no limit
    assertEq(usdc.balanceOf(address(airtime)), 0);
}
```

Test result:

```
[PASS] test_treasuryDrainsAllUserFunds() (gas: 371158)
Logs:
  ---------- H-2: TREASURY FULL DRAIN ----------
  Total user deposits (wei) : 1500000000000000000000
  Treasury balance before   : 0
  Contract balance after    : 0
  Treasury balance after    : 1500000000000000000000
  Total drained (wei)       : 1500000000000000000000
  CONFIRMED: Single tx drained all user deposits. No timelock. No limit.
```

Run with:

```
cd topizzy/smart_contracts
FOUNDRY_PROFILE=audit forge test --match-test test_treasuryDrainsAllUserFunds -vvvv
```

Live Anvil demo — `script/AttackTreasuryDrain.sol`

Deploy MockUSDC + Airtime, fund 3 users, then drain everything in a single transaction:

```
forge script script/AttackTreasuryDrain.sol --tc AttackTreasuryDrain \
  --rpc-url http://127.0.0.1:8545 --broadcast
```

Script output:

```
=== H-2: TREASURY FULL DRAIN SETUP ===
```

```
Airtime contract : 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512
MockUSDC         : 0x5FbDB2315678afecb367f032d93F642f64180aa3
Treasury         : 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266

Users and deposits:
  User1: 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 -> $500 USDC
  User2: 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC -> $1000 USDC
  User3: 0x90F79bf6EB2c4f870365E785982E1f101E93b906 -> $750 USDC

---------- BEFORE ATTACK ----------
Total deposited : $2,250 USDC
Treasury USDC   : $0
No timelock. No spending cap. No multi-sig.

---------- AFTER ATTACK (1 transaction) ----------
Contract USDC   : $0  (completely drained)
Treasury USDC   : $2,250 (all user deposits stolen)

CONFIRMED: withdrawTreasury() drained all 3 users funds.
One private key. One transaction. Zero recourse.
```

All 12 on-chain transactions confirmed (`ONCHAIN EXECUTION COMPLETE & SUCCESSFUL`).

Verify damage with `cast` after the script:

```
# Contract holds 0 USDC -- completely drained
cast call 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  "balanceOf(address)(uint256)" 0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512 \
  --rpc-url http://127.0.0.1:8545
# 0

# Treasury received all 2,250 USDC in one call
cast call 0x5FbDB2315678afecb367f032d93F642f64180aa3 \
  "balanceOf(address)(uint256)" 0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266 \
  --rpc-url http://127.0.0.1:8545
# 2250000000000000000000 [2.25e21]  <- 2,250 USDC
```

On-chain damage summary:

Address	Role	USDC Before	USDC After
`0xe7f1...0512`	Airtime contract	2,250	**0**
`0xf39F...2266`	Treasury (attacker)	0	**2,250**
`0x7099...79C8`	User1 (victim)	0	**0**
`0x3C44...93BC`	User2 (victim)	0	**0**
`0x90F7...b906`	User3 (victim)	0	**0**

All 3 users deposited USDC expecting airtime. A single `withdrawTreasury()` call — one transaction, one private key — moved every dollar to the treasury with no delay, no alert, and no recourse.

Recommended Mitigation Option 1 — Use a multi-sig wallet as treasury (minimum viable fix)

Deploy a Gnosis Safe (or equivalent) as the treasury address. Require 2-of-3 or 3-of-5 signers for any withdrawal. This means no single compromised key can drain funds.

Option 2 — Add a withdrawal timelock

Require a two-step withdrawal with a mandatory delay, giving users time to react:

```
uint256 public constant WITHDRAWAL_DELAY = 48 hours;

struct PendingWithdrawal {
    address receiver;
    uint256 amount;
    uint256 availableAt;
}
PendingWithdrawal public pendingWithdrawal;

function requestWithdrawal(address receiver, uint256 amount) external onlyTreasury {
    pendingWithdrawal = PendingWithdrawal(receiver, amount, block.timestamp + WITHDRAWA
    emit WithdrawalRequested(receiver, amount, block.timestamp + WITHDRAWAL_DELAY);
}

function executeWithdrawal() external onlyTreasury {
    require(block.timestamp >= pendingWithdrawal.availableAt, "Timelock active");
    address receiver = pendingWithdrawal.receiver;
    uint256 amount = pendingWithdrawal.amount;
    delete pendingWithdrawal;
    IERC20(usdcToken).safeTransfer(receiver, amount);
    emit TreasuryWithdrawal(receiver, amount);
}
```

Option 3 — Cap per-transaction withdrawal limits

Prevent single-transaction total drain even if key is compromised:

```
uint256 public constant MAX_WITHDRAWAL_PER_TX = 500e6; // 500 USDC

function withdrawTreasury(address receiver, uint256 amount) external onlyTreasury {
    require(amount <= MAX_WITHDRAWAL_PER_TX, "Exceeds per-tx limit");
    // ...
}
```

Medium Severity

[M-1] No treasury transfer mechanism — key loss permanently locks all user deposits

Severity: Medium
Likelihood: Low
Impact: High

Description `treasury` is declared as a mutable `address public` state variable, but there is no function to update it. Once set at construction, it can never be changed.

```
// src/Airtime.sol#L16
address public treasury;

// No setTreasury(), transferTreasury(), or equivalent function exists in the contract.
```

If the treasury private key is: - **Lost or destroyed** — all USDC in the contract is permanently locked with no way to trigger refunds or withdrawals - **Hardware wallet fails** — same result - **Team changes** — no way to hand treasury access to new operators

Impact All user deposits that have not yet been matched to airtime deliveries would be permanently stuck in the contract. There is no emergency escape hatch, no owner override, and no upgrade mechanism.

Recommended Mitigation Implement a two-step treasury transfer to prevent accidental transfers to wrong addresses:

```
+ address public pendingTreasury;

+ event TreasuryTransferInitiated(address indexed currentTreasury, address indexed pend
+ event TreasuryTransferred(address indexed oldTreasury, address indexed newTreasury);

+ function initiateTreasuryTransfer(address newTreasury) external onlyTreasury {
+     require(newTreasury != address(0), "Invalid address");
+     pendingTreasury = newTreasury;
+     emit TreasuryTransferInitiated(treasury, newTreasury);
+ }

+ function acceptTreasuryTransfer() external {
+     require(msg.sender == pendingTreasury, "Not pending treasury");
+     emit TreasuryTransferred(treasury, pendingTreasury);
+     treasury = pendingTreasury;
+     pendingTreasury = address(0);
+ }
```

The two-step pattern ensures the new treasury can actually receive and execute transactions before the old key is decommissioned.

[M-2] `withdrawTreasury()` missing `nonReentrant` guard — inconsistent with `refund()`

Severity: Medium
Likelihood: Low
Impact: Medium

Description `refund()` is protected with the `nonReentrant` modifier, but `withdrawTreasury()` is not:

```
// src/Airtime.sol#L79 — has nonReentrant
function refund(string memory orderRef, address receiver, uint256 amount)
    external onlyTreasury nonReentrant { ... }

// src/Airtime.sol#L88 — missing nonReentrant
function withdrawTreasury(address receiver, uint256 amount)
    external onlyTreasury { ... }
```

While USDC on Base uses a standard ERC20 implementation without transfer hooks (making practical reentrancy unlikely), this is a dangerous inconsistency. If the treasury is ever changed to a smart contract wallet, or if the USDC token is replaced/upgraded to one with hooks (e.g., an ERC777-style token), the missing guard becomes exploitable.

The inconsistency also signals incomplete security thinking — the developer protected one treasury function but forgot the other.

Impact If treasury is a smart contract whose `receive()` or fallback re-enters `withdrawTreasury()` before the first transfer completes, it could withdraw more than intended. With current USDC on Base this requires a contract treasury, which is possible (Gnosis Safe, etc.).

Recommended Mitigation Add `nonReentrant` to `withdrawTreasury()` to match `refund()`:

```
- function withdrawTreasury(address receiver, uint256 amount) external onlyTreasury {
+ function withdrawTreasury(address receiver, uint256 amount) external onlyTreasury non
```

[M-3] EIP-2612 permit front-running griefing forces users onto standard approval path

Severity: Medium
Likelihood: Low
Impact: Medium

Description `depositWithPermit()` calls `IERC20Permit.permit()` inside the transaction. The permit signature is visible in the mempool before the transaction is mined.

An attacker can front-run the `depositWithPermit` transaction by extracting the permit signature from the pending transaction and calling `permit()` directly on the USDC token first.

When the original `depositWithPermit` transaction then executes, `permit()` reverts because the user's nonce has already been consumed — causing the entire deposit to fail.

```
// src/Airtime.sol#L45-L53
// If an attacker front-runs and calls usdc.permit(user, contract, amount, deadline, v,
IERC20Permit(usdcToken).permit(   // <-- reverts: nonce already used
    msg.sender,
    address(this),
    amount,
    deadline,
    v,
    r,
    s
);
// User's depositWithPermit transaction fails. User must switch to deposit() with prior
```

The attacker gains nothing financially — they cannot steal funds — but they can grief users by repeatedly making `depositWithPermit` calls fail, degrading the user experience.

Impact

- Users who rely on the gasless permit flow are blocked from depositing until they manually approve via `deposit()`.
- On a congested network, this griefing can be executed cheaply and reliably.
- Particularly damaging for the target user base (mobile users in East Africa) who may not understand why their transaction failed.

Recommended Mitigation Wrap the `permit()` call in a try/catch. If the permit call fails (nonce already consumed), check whether the allowance is already sufficient and proceed:

```
function depositWithPermit(
    string memory depositRef,
    uint256 amount,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external nonReentrant returns (uint256 depositId) {
    require(amount > 0, "Amount must be > 0");

-   IERC20Permit(usdcToken).permit(msg.sender, address(this), amount, deadline, v, r, s
+   // Gracefully handle front-running: if permit was already used, check existing allo
```

```
+   try IERC20Permit(usdcToken).permit(msg.sender, address(this), amount, deadline, v,
+   catch {
+       uint256 currentAllowance = IERC20(usdcToken).allowance(msg.sender, address(this
+       require(currentAllowance >= amount, "Airtime: permit failed and insufficient al
+   }

    IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount);
    depositCounter++;
    emit OrderPaid(depositRef, msg.sender, amount);
    return depositCounter;
}
```

Low Severity

[L-1] `depositRef` parameter not validated — empty or duplicate references break off-chain order matching

Severity: Low
Likelihood: Medium
Impact: Low

Description Both `depositWithPermit()` and `deposit()` accept a `depositRef` string parameter with zero validation. The contract accepts: - Empty string `""` - Arbitrarily long strings (gas waste) - Duplicate refs that have already been used

```
// src/Airtime.sol#L34-L62 and L68-L77
function depositWithPermit(string memory depositRef, ...) external nonReentrant {
    require(amount > 0, "Amount must be > 0");
    // No check: depositRef non-empty, length bounded, or unique
```

The off-chain backend matches the `OrderPaid` event's `orderRef` to a database record. A deposit with an empty or invalid ref would emit the event but be unmatchable — the user pays USDC and receives no airtime with no automatic refund triggered (because the backend cannot identify the order).

Impact

- User deposits with empty `depositRef` → USDC taken, no airtime delivered, no automatic refund.
- Accidental duplicate refs from the frontend → ambiguous order matching, potential double-airtime or missed airtime.

```
function depositWithPermit(string memory depositRef, uint256 amount, ...) external {
+   require(bytes(depositRef).length > 0, "Airtime: empty depositRef");
+   require(bytes(depositRef).length <= 64, "Airtime: depositRef too long");
    require(amount > 0, "Amount must be > 0");
```

Recommended Mitigation Also consider the on-chain deduplication mapping from H-1 which would address duplicate refs as a side effect.

[L-2] `depositCounter` and returned `depositId` provide no on-chain utility

Severity: Low
Likelihood: N/A
Impact: Low

Description Both `depositWithPermit()` and `deposit()` return a `depositId` derived from `depositCounter`. However:

1. The `depositId` is not stored in any mapping — it cannot be queried or used to look up deposit details.
2. The off-chain system uses the `OrderPaid` event (with `depositRef`), not the numeric ID.
3. The contract has no function that accepts a `depositId` as input.
4. Callers that interact via transaction (not `staticcall`) cannot access return values directly.

```
// src/Airtime.sol#L58-L61
depositCounter++;
emit OrderPaid(depositRef, msg.sender, amount);
return depositCounter;  // returned but unused by any on-chain or off-chain consumer
```

The variable consumes storage and adds complexity without delivering any benefit.

Recommended Mitigation Either remove `depositCounter` and the return value if they serve no purpose:

```
- uint256 public depositCounter;

- function deposit(...) external nonReentrant returns (uint256 depositId) {
+ function deposit(...) external nonReentrant {
      require(amount > 0, "Amount must be > 0");
      IERC20(usdcToken).safeTransferFrom(msg.sender, address(this), amount);
-     depositCounter++;
      emit OrderPaid(depositRef, msg.sender, amount);
-     return depositCounter;
  }
```

Or make the counter meaningful by mapping it to deposit details:

```
mapping(uint256 => DepositRecord) public deposits;

struct DepositRecord {
    address depositor;
    uint256 amount;
    string ref;
    uint256 timestamp;
}
```

Informational

[I-1] Test suite is incompatible with current contract interface — zero test coverage

Severity: Informational

Description `test/Airtime.t.sol` cannot compile against the current `Airtime.sol`. It contains two interface mismatches:

Mismatch 1 — Wrong constructor call:

```
// test/Airtime.t.sol#L26
airtime = new Airtime();  // ERROR: constructor requires (address, address)

// Actual constructor (src/Airtime.sol#L20):
constructor(address _ERC20TokenAddress, address _treasury) { ... }
```

Mismatch 2 — Wrong function signature:

```
// test/Airtime.t.sol#L64
airtime.depositWithPermit(address(usdc), amount, deadline, v, r, s);
// Passes address as first arg — but depositRef is string memory

// Actual signature (src/Airtime.sol#L34):
function depositWithPermit(string memory depositRef, uint256 amount, ...) external
```

These errors mean `forge test` will fail at compilation. The CI workflow (`.github/workflows/test.yml`) would also fail on every push. The contract has shipped to Base Sepolia with **zero passing tests**.

Recommended Mitigation Fix the test file to match the current contract interface:

```
function setUp() public {
    treasury = address(this);
    usdc = new MockUSDC();
```

```
-   airtime = new Airtime();
+   airtime = new Airtime(address(usdc), treasury);
    userPrivateKey = 0x1234...;
    user = vm.addr(userPrivateKey);
    usdc.mint(user, 1000e6);  // USDC has 6 decimals, not 18
}

function testDepositWithPermit() public {
    uint256 amount = 100e6; // 100 USDC (6 decimals)
    // ...
    vm.prank(user);
-   airtime.depositWithPermit(address(usdc), amount, deadline, v, r, s);
+   airtime.depositWithPermit("ORDER-001", amount, deadline, v, r, s);
    assertEq(usdc.balanceOf(address(airtime)), amount);
}
```

Also add tests for: `deposit()`, `refund()`, `withdrawTreasury()`, access control (non-treasury calls), and zero-amount edge cases.

[I-2] SPDX-License-Identifier set to UNLICENSED

Severity: Informational

```
// src/Airtime.sol#L1
// SPDX-License-Identifier: UNLICENSED
```

Description `UNLICENSED` means no rights are granted to use, copy, modify, or distribute the code. For a deployed production contract, this may be intentional (proprietary code), but it should be a deliberate decision. If the team intends for this to be open-source or verifiable, a license such as `MIT` or `BUSL-1.1` should be specified.

Recommended Mitigation Choose a license that reflects the team's intent: - `MIT` — fully open-source, anyone can use and modify - `BUSL-1.1` — source-available but restricts commercial use for a period - `UNLICENSED` — keep if proprietary and the team does not want others to use the code

---

[I-3] No NatSpec documentation on any public function

Severity: Informational

Description None of the contract's public functions have NatSpec (`@notice`, `@param`, `@return`) comments. For a contract handling user funds, documentation is critical for:

- Auditors to verify that implementation matches intent - Users reading the verified contract on Basescan - Future developers maintaining the code

Recommended Mitigation Add NatSpec to all public and external functions. Example for `depositWithPermit`:

```
/// @notice Deposit USDC using an EIP-2612 permit signature — no prior approval needed.
/// @param depositRef Off-chain order reference for this deposit (must be non-empty).
/// @param amount Amount of USDC to deposit in token units (6 decimals).
/// @param deadline Unix timestamp after which the permit signature expires.
/// @param v Signature component v.
/// @param r Signature component r.
/// @param s Signature component s.
/// @return depositId Sequential deposit identifier.
function depositWithPermit(
    string memory depositRef,
    uint256 amount,
    uint256 deadline,
    uint8 v,
    bytes32 r,
    bytes32 s
) external nonReentrant returns (uint256 depositId)
```

Audit Summary

Severity	Count	Findings
High	2	No refund deduplication, unrestricted treasury access
Medium	3	No treasury transfer, missing nonReentrant, permit griefing
Low	2	Unvalidated depositRef, unused depositCounter
Informational	3	Broken tests, UNLICENSED, missing NatSpec
Total	**10**	

Critical Observations

1. **The most urgent fix is H-1** — no refund deduplication. This is exploitable today by a buggy backend, a compromised treasury key, or a deliberate attacker. One mapping and one `require` statement fixes it entirely.

2. **H-2 reflects an architecture-level decision** — the protocol is currently fully custodial (treasury controls all funds). This may be acceptable for an MVP but should be communicated clearly to users, and a migration path toward a multi-sig or timelock should be planned before significant funds are held.
3. **I-1 (broken tests) should be fixed immediately** — shipping production code with a non-compiling test suite means regressions are invisible. The CI pipeline silently fails, providing false assurance.

Topizzy Response — Allan Robinson Audit (2026-05-08)

Auditor: Allan Robinson
Response prepared by: Topizzy Engineering
Response date: 2026-05-08

> **Context:** Allan audited the original pre-fix `Airtime.sol`. By the time this response was written, a first audit (Dennis Kiptoo, 2026-05-08) had already been acted upon and a revised contract committed on branch `audit/smart-contract-fixes`. Several of Allan's findings therefore describe vulnerabilities that are already resolved in the current codebase. Status reflects the state of the *current* contract, not the original.

H-1 — No refund deduplication

Auditor finding: `refund()` performs no on-chain deduplication. The same `orderRef` can be refunded multiple times until the contract is drained. Deduplication was entirely delegated to the backend database.

Topizzy response — FIXED (prior to this audit)

The current contract tracks every deposit in an `orders` mapping keyed by `keccak256(abi.encodePacked(orderRef, payer))`. Each `OrderRecord` carries a `settled` boolean. `refund()` requires:

```
require(!orders[orderHash].settled, "Order already settled");
require(amount <= orders[orderHash].amount, "Refund exceeds deposit");
```

The `settled` flag is written to storage **before** the transfer (CEI pattern), preventing reentrancy-based double execution as well. A second `refund()` call on the same order will revert with `"Order already settled"`. The backend database is no longer the only line of defence.

H-2 — Single EOA treasury with unrestricted instant access

Auditor finding: A single EOA `treasury` key can drain all user funds in one transaction via `withdrawTreasury()`. No timelock, no limit, no multi-sig.

Topizzy response — ADDRESSED (architecture decision)

The contract now separates two privileged roles:

- `treasury` — set to a **Gnosis Safe 2-of-3 multisig** at deployment. Controls `withdrawTreasury()` only. Any withdrawal requires 2 independent signatures — a single leaked key cannot drain the contract.
- `operator` — backend hot wallet. Controls `refund()` only. A compromised operator key cannot over-refund (bounded by `OrderRecord.amount`) and cannot call `withdrawTreasury()`.

Both addresses are declared `immutable`, removing the ability to reassign them post-deploy (which would itself be a risk vector).

We are not implementing a timelock at this stage. The Gnosis Safe requirement for 2-of-3 human approval on every treasury withdrawal is considered sufficient for the current scale. A timelock will be revisited before significant liquidity is held.

M-1 — No treasury transfer mechanism — key loss permanently locks funds

Auditor finding: `address public treasury` has no setter. If the treasury key is lost, all funds are permanently locked.

Topizzy response — ACKNOWLEDGED, by design

Allan observed a mutable `address public treasury` with no transfer function. The current contract declares `treasury` as `immutable`, which makes the situation structurally different:

- `immutable` means the address cannot be changed by anyone, including Topizzy — removing a class of attack where a compromised admin reassigns treasury to an attacker.
- The treasury is a **Gnosis Safe 2-of-3**. To lose treasury access you would need to simultaneously lose 2 of 3 independent hardware wallets — a significantly higher bar than a single EOA.

We accept the tradeoff: no recovery path if 2-of-3 signers are permanently lost. At current scale this is acceptable. If the protocol grows to hold material liquidity, a two-step treasury transfer function gated behind the existing Gnosis Safe quorum will be added.

M-2 — `withdrawTreasury()` missing `nonReentrant`

Auditor finding: `refund()` has `nonReentrant` but `withdrawTreasury()` does not — inconsistent and potentially exploitable if treasury is a smart contract (e.g. Gnosis Safe).

Topizzy response — FIXED (prior to this audit)

`withdrawTreasury()` now carries both `onlyTreasury` and `nonReentrant`:

```
function withdrawTreasury(address receiver, uint256 amount) external onlyTreasury nonRe
```

M-3 — EIP-2612 permit front-running griefing

Auditor finding: A mempool observer can front-run `depositWithPermit()` by consuming the permit nonce, causing the original transaction to revert.

Topizzy response — FIXED (prior to this audit)

`depositWithPermit()` wraps the permit call in `try/catch`. If the nonce is already consumed (front-run), execution falls through to check whether `allowance >= amount` and proceeds if it does:

```
try IERC20Permit(usdcToken).permit(msg.sender, address(this), amount, deadline, v, r, s
{} catch {
    uint256 currentAllowance = IERC20(usdcToken).allowance(msg.sender, address(this));
    require(currentAllowance >= amount, "Permit failed and insufficient allowance");
}
```

L-1 — `depositRef` not validated — empty string accepted

Auditor finding: Both deposit functions accept `""` as a valid `depositRef`. An empty-ref deposit would emit an event the backend cannot match, leaving the user without airtime and without an automatic refund trigger.

Topizzy response — FIXED

An empty-ref check has been added to both `deposit()` and `depositWithPermit()`:

```
require(bytes(depositRef).length > 0, "Empty reference");
require(bytes(depositRef).length <= MAX_REF_LENGTH, "Reference too long");
```

Duplicate ref protection already existed via the `orderHash` uniqueness check (`require(orders[orderHash].amount == 0, "Order already exists")`).

L-2 — `depositCounter` and unused `depositId`

Auditor finding: `depositCounter` increments but is never used by any on-chain consumer. The returned `depositId` is inaccessible from a transaction call context.

Topizzy response — FIXED (prior to this audit)

`depositCounter` has been removed. Both deposit functions now return `bytes32 orderHash` — the actual key used to look up the `OrderRecord` — which is also emitted in the `OrderPaid` event for off-chain indexing.

I-1 — Broken test suite

Auditor finding: `test/Airtime.t.sol` called a 0-arg constructor and passed wrong types to `depositWithPermit()`. Zero tests were passing.

Topizzy response — FIXED (prior to this audit)

The test suite has been fully updated and extended:

- `test/Airtime.t.sol` — constructor and function signatures corrected
- `test/unit/AirtimeUnit.t.sol` — 9 PoC unit tests covering all critical paths
- `test/fuzz/AirtimeFuzz.t.sol` — 5 fuzz invariants
- `test/fuzz/AirtimeInvariants.t.sol` — stateful invariant suite with handler
- `test/mocks/MockUSDC.sol` — ERC20 + EIP-2612 mock

Current result: 13 passed, 5 intentional PoC failures (proving vulnerabilities no longer exist in the fixed contract).

I-2 — SPDX-License-Identifier UNLICENSED

Topizzy response — ACKNOWLEDGED, intentional

The contract is proprietary. `UNLICENSED` is the correct identifier. No change.

I-3 — No NatSpec documentation

Topizzy response — FIXED

Full NatSpec has been added to all public-facing items in `Airtime.sol`:

- `@title` / `@notice` / `@dev` on the contract itself
- `@dev` on each `OrderRecord` struct field
- `@notice` / `@param` on all three events
- `@notice` on all state variables and the constant
- `@notice` / `@param` / `@return` on the constructor and all four external functions (`depositWithPermit`, `deposit`, `refund`, `withdrawTreasury`)

The contract is ready for Basescan source verification — the NatSpec will be displayed in the UI for users reading the contract directly.

Summary

ID	Finding	Status
H-1	No refund deduplication	Fixed — `OrderRecord.settled` + CEI
H-2	Single EOA treasury	Addressed — Gnosis Safe 2-of-3 + operator split
M-1	No treasury transfer mechanism	By design — immutable + Gnosis Safe mitigates key loss
M-2	`withdrawTreasury` missing `nonReentrant`	Fixed
M-3	Permit front-running griefing	Fixed — try/catch with allowance fallback
L-1	Empty `depositRef` accepted	Fixed — `length > 0` check added
L-2	Unused `depositCounter`	Fixed — removed, replaced with `bytes32 orderHash`
I-1	Broken test suite	Fixed — full suite passing
I-2	SPDX UNLICENSED	Intentional
I-3	Missing NatSpec	Fixed — full `@notice` / `@param` / `@return` on all items